contents

front matter

foreword

preface

acknowledgments

about this book

about the author

about the cover illustration

Part 1 Cloud native fundamentals

1 Introduction to cloud native

1.1 What is cloud native?

The Three Ps of Cloud Native

1.2 The cloud and the cloud computing model

Infrastructure as a Service (IaaS)

Container as a Service (CaaS)

Platform as a Service (PaaS)

Function as a Service (FaaS)

Software as a Service (SaaS)

1.3 Properties of cloud native applications

Scalability

Loose coupling

Resilience

Observability

Manageability

1.4 Culture and practices supporting cloud native

Automation

Continuous delivery

DevOps

1.5 Is the cloud your best option?

Speed

Resilience

Scale

Cost

1.6 Cloud native topologies

Containers

Orchestration

Serverless

1.7 Architectures for cloud native applications

From multi-tiered to microservices architectures and beyond

Service-based architecture for cloud native applications

2 Cloud native patterns and technologies

2.1 Cloud native development principles: 12 Factors and beyond

One codebase, one application

API first

Dependency management

Design, build, release, run

Configuration, credentials, and code

Logs

Disposability

Backing services

Environment parity

Administrative processes

Port binding

Stateless processes

Concurrency

Telemetry

Authentication and authorization

2.2 Building cloud native applications with Spring

Overview of the Spring landscape

Building a Spring Boot application

2.3 Containerizing applications with Docker

Introducing Docker: Images and containers

Running a Spring application as a container

2.4 Managing containers with Kubernetes

Introducing Kubernetes: Deployments, Pods, and Services

Running a Spring application on Kubernetes

2.5 Polar Bookshop: A cloud native application

Understanding the requirements of the system

Exploring patterns and technologies used in the project

Part 2 Cloud native development

3 Getting started with cloud native development

3.1 Bootstrapping a cloud native project

One codebase, one application

Dependency management with Gradle and Maven

3.2 Working with embedded servers

Executable JARs and embedded servers

Understanding the thread-per-request model

Configuring the embedded Tomcat

3.3 Building a RESTful application with Spring MVC

REST API first, business logic later

Implementing a REST API with Spring MVC

Data validation and error handling

Evolving APIs for future requirements

3.4 Testing a RESTful application with Spring

Unit tests with JUnit

Integration tests with @SpringBootTest

Testing REST controllers with @WebMvcTest

Testing the JSON serialization with @JsonTest

3.5 Deployment pipeline: Build and test

Understanding the commit stage of the deployment pipeline

Implementing the commit stage with GitHub Actions

4 Externalized configuration management

4.1 Configuration in Spring: Properties and profiles

Properties: Key/value pairs for configuration

Profiles: Feature flags and configuration groups

4.2 Externalized configuration: One build, multiple configurations

Configuring an application through command-line arguments

Configuring an application through JVM system properties

Configuring an application through environment variables

4.3 Centralized configuration management with Spring Cloud Config Server

Using Git to store your configuration data

Setting up a configuration server

Making the configuration server resilient

Understanding the configuration server REST API

4.4 Using a configuration server with Spring Cloud Config Client

Setting up a configuration client

Making the configuration client resilient

Refreshing configuration at runtime

5 Persisting and managing data in the cloud

5.1 Databases for cloud native systems

Data services in the cloud

Running PostgreSQL as a container

5.2 Data persistence with Spring Data JDBC

Connecting to a database with JDBC

Defining persistent entities with Spring Data

Enabling and configuring JDBC auditing

Data repositories with Spring Data

5.3 Testing data persistence with Spring and Testcontainers

Configuring Testcontainers for PostgreSQL

Testing data persistence with @DataJdbcTest and Testcontainers

Integration tests with @SpringBootTest and Testcontainers

5.4 Managing databases in production with Flyway

Understanding Flyway: Version control for your database

Initializing a database schema with Flyway

Evolving a database with Flyway

6 Containerizing Spring Boot

6.1 Working with container images on Docker

Understanding container images

Creating images with Dockerfiles

Publishing images on GitHub Container Registry

6.2 Packaging Spring Boot applications as container images

Preparing Spring Boot for containerization

Containerizing Spring Boot with Dockerfiles

Building container images for production

Containerizing Spring Boot with Cloud Native Buildpacks

6.3 Managing Spring Boot containers with Docker Compose

Using Docker Compose to manage the container life cycle

Debugging Spring Boot containers

6.4 Deployment pipeline: Package and publish

Building release candidates in the commit stage

Publishing container images with GitHub Actions

7 Kubernetes fundamentals for Spring Boot

7.1 Moving from Docker to Kubernetes

Working with a local Kubernetes cluster

Managing data services in a local cluster

7.2 Kubernetes Deployments for Spring Boot

From containers to Pods

Controlling Pods with Deployments

Creating a Deployment for a Spring Boot application

7.3 Service discovery and load balancing

Understanding service discovery and load balancing

Client-side service discovery and load balancing

Server-side service discovery and load balancing

Exposing Spring Boot applications with Kubernetes Services

7.4 Scalability and disposability

Ensuring disposability: Fast startup

Ensuring disposability: Graceful shutdown

Scaling Spring Boot applications

7.5 Local Kubernetes development with Tilt

Inner development loop with Tilt

Visualizing your Kubernetes workloads with Octant

7.6 Deployment pipeline: Validate Kubernetes manifests

Validating Kubernetes manifests in the commit stage

Automating Kubernetes manifests validation with GitHub Actions

Part 3 Cloud native distributed systems

8 Reactive Spring: Resilience and scalability

8.1 Asynchronous and non-blocking architectures with Reactor and Spring

From thread-per-request to event loop

Project Reactor: Reactive streams with Mono and Flux

Understanding the Spring reactive stack

8.2 Reactive servers with Spring WebFlux and Spring Data R2DBC

Bootstrapping a reactive application with Spring Boot

Persisting data reactively with Spring Data R2DBC

Implementing the business logic with reactive streams

Exposing a REST API with Spring WebFlux

8.3 Reactive clients with Spring WebClient

Service-to-service communication in Spring

Understanding how to exchange data

Implementing REST clients with WebClient

8.4 Resilient applications with Reactive Spring

Timeouts

Retries

Fallbacks and error handling

8.5 Testing reactive applications with Spring, Reactor, and Testcontainers

Testing REST clients with a mock web server

Testing data persistence with @DataR2dbcTest and Testcontainers

Testing REST controllers with @WebFluxTest

9 API gateway and circuit breakers

9.1 Edge servers and Spring Cloud Gateway

Bootstrapping an edge server with Spring Cloud Gateway

Defining routes and predicates

Processing requests and responses through filters

9.2 Fault tolerance with Spring Cloud Circuit Breaker and Resilience4J

Introducing circuit breakers with Spring Cloud Circuit Breaker

Configuring a circuit breaker with Resilience4J

Defining fallback REST APIs with Spring WebFlux

Combining circuit breakers, retries, and time limiters

9.3 Request rate limiting with Spring Cloud Gateway and Redis

Running Redis as a container

Integrating Spring with Redis

Configuring a request rate limiter

9.4 Distributed session management with Redis

Handling sessions with Spring Session Data Redis

9.5 Managing external access with Kubernetes Ingress

Understanding Ingress API and Ingress Controller

Working with Ingress objects

10 Event-driven applications and functions

10.1 Event-driven architectures

Understanding the event-driven models

Using the pub/sub model

10.2 Message brokers with RabbitMQ

Understanding AMQP for messaging systems

Using RabbitMQ for publish/subscribe communications

10.3 Functions with Spring Cloud Function

Using the functional paradigm in Spring Cloud Function

Composing and integrating functions: REST, serverless, data streams

Writing integration tests with @FunctionalSpringBootTest

10.4 Processing messages with Spring Cloud Stream

Configuring the integration with RabbitMQ

Binding functions to message channels

Writing integration tests with a test binder

Making messaging resilient to failures

10.5 Producing and consuming messages with Spring Cloud Stream

Implementing event consumers, and the problem of idempotency

Implementing event producers, and the problem of atomicity

11 Security: Authentication and SPA

11.1 Understanding the Spring Security fundamentals

11.2 Managing user accounts with Keycloak

Defining a security realm

Managing users and roles

11.3 Authentication with OpenID Connect, JWT, and Keycloak

Authenticating users with OpenID Connect

Exchanging user information with JWT

Registering an application in Keycloak

11.4 Authenticating users with Spring Security and OpenID Connect

Adding the new dependencies

Configuring the integration between Spring Security and Keycloak

Basic Spring Security configuration

Inspecting the authenticated user context

Configuring user logout in Spring Security and Keycloak

11.5 Integrating Spring Security with SPAs

Running an Angular application

Controlling the authentication flow

Protecting against Cross-Site Request Forgery

11.6 Testing Spring Security and OpenID Connect

Testing OIDC authentication

Testing CSRF

12 Security: Authorization and auditing

12.1 Authorization and roles with Spring Cloud Gateway and OAuth

Token relay from Spring Cloud Gateway to other services

Customizing tokens and propagating user roles

12.2 Protecting APIs with Spring Security and OAuth2 (imperative)

Securing Spring Boot as an OAuth2 Resource Server

Role-based access control with Spring Security and JWT

Testing OAuth2 with Spring Security and Testcontainers

12.3 Protecting APIs with Spring Security and OAuth2 (reactive)

Securing Spring Boot as an OAuth2 Resource Server

Testing OAuth2 with Spring Security and Testcontainers

12.4 Protecting and auditing data with Spring Security and Spring Data

Auditing data with Spring Security and Spring Data JDBC

Testing data auditing with Spring Data and @WithMockUser

Protecting user data with Spring Security and Spring Data R2DBC

Testing data auditing and protection with @WithMockUser and Spring Data R2DBC

Part 4 Cloud native production

13 Observability and monitoring

13.1 Logging with Spring Boot, Loki, and Fluent Bit

Logging with Spring Boot

Managing logs with Loki, Fluent Bit, and Grafana

13.2 Health probes with Spring Boot Actuator and Kubernetes

Defining health probes for Spring Boot applications using Actuator

Configuring health probes in Spring Boot and Kubernetes

13.3 Metrics and monitoring with Spring Boot Actuator, Prometheus, and Grafana

Configuring metrics with Spring Boot Actuator and Micrometer

Monitoring metrics with Prometheus and Grafana

Configuring Prometheus metrics in Kubernetes

13.4 Distributed tracing with OpenTelemetry and Tempo

Managing traces with Tempo and Grafana

Configuring tracing in Spring Boot with OpenTelemetry

13.5 Application management and monitoring with Spring Boot Actuator

Monitoring Flyway migrations in Spring Boot

Exposing application information

Generating and analyzing heap dumps

14 Configuration and secrets management

14.1 Configuring applications on Kubernetes

Securing the configuration server with Spring Security

Refreshing configuration at runtime with Spring Cloud Bus

Managing secrets with Spring Cloud Config

Disabling Spring Cloud Config

14.2 Using ConfigMaps and Secrets in Kubernetes

Configuring Spring Boot with ConfigMaps

Storing sensitive information with Secrets (or not)

Refreshing configuration at runtime with Spring Cloud Kubernetes

14.3 Configuration management with Kustomize

Using Kustomize to manage and configure Spring Boot applications

Managing Kubernetes configuration for multiple environments with Kustomize

Defining a configuration overlay for staging

Customizing environment variables

Customizing ConfigMaps

Customizing image name and version

Customizing the number of replicas

15 Continuous delivery and GitOps

15.1 Deployment pipeline: Acceptance stage

Versioning release candidates for continuous delivery

Understanding the acceptance stage of the deployment pipeline

Implementing the acceptance stage with GitHub Actions

15.2 Configuring Spring Boot for production

Defining a configuration overlay for production

Configuring CPU and memory for Spring Boot containers

Deploying Spring Boot in production

15.3 Deployment pipeline: Production stage

Understanding the production stage of the deployment pipeline

Implementing the production stage with GitHub Actions

15.4 Continuous deployment with GitOps

Implementing GitOps with Argo CD

Putting it all together

16 Serverless, GraalVM, and Knative

16.1 Native images with Spring Native and GraalVM

Understanding GraalVM and native images

Introducing GraalVM support for Spring Boot with Spring Native

Compiling Spring Boot applications as native images

16.2 Serverless applications with Spring Cloud Function

Building serverless applications with Spring Cloud Function

Deployment pipeline: Build and publish

Deploying serverless applications on the cloud

16.3 Deploying serverless applications with Knative

Setting up a local Knative platform

Deploying applications with the Knative CLI

Deploying applications with the Knative manifests

appendix A Setting up your development environment

appendix B Kubernetes in production with DigitalOcean

index